

1. OR Gate Implementation Using M-P Neuron

Source code and Output:-

```
import numpy as np

def linear_threshold_gate(dot_product, threshold):

    return 1 if dot_product >= threshold else 0

inputs = np.array([

    [0, 0],

    [0, 1],

    [1, 0],

    [1, 1]

])

weights = np.array([1, 1])

dot_products = np.dot(inputs, weights)

T = 1

for i in range(0,4):

    activation = linear_threshold_gate(dot_products[i],T)

    print(f"Inputs: {inputs[i]} → Activation: {activation}")
```

Inputs: [0 0] → Activation: 0

Inputs: [0 1] → Activation: 1

Inputs: [1 0] → Activation: 1

Inputs: [1 1] → Activation: 1

2. AND Gate Implementation Using M-P Neuro

3. Source code and Output:-

```
import numpy as np

inputs = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

weights = np.array([1,1])

def linear_threshold_gate(dot_product,threshold):
    return 1 if dot_product >= threshold else 0

dot_products = np.dot(inputs,weights)

T = 2

for i in range(0,4):
    activation = linear_threshold_gate(dot_products[i],T)
    print(f"Inputs: {inputs[i]} → Activation: {activation}")
```

Inputs: [0 0] → Activation: 0

Inputs: [0 1] → Activation: 0

Inputs: [1 0] → Activation: 0

Inputs: [1 1] → Activation: 1

4. NOT Gate Implementation Using M-P Neuron Source code and Output:-

```
inputs = np.array([0,1])

weights = np.array([-1])
```

```
T = 0
```

```
for i in range(0,2):
```

```
    activation = linear_threshold_gate(np.dot(inputs[i],weights),T)
```

```
    print(f"Inputs: {inputs[i]} → Activation: {activation}")
```

Inputs: 0 → Activation: 1

Inputs: 1 → Activation: 0

1) Linear Activation Function

Source code and Output:-

```
def linear(x):  
    return x  
plt.figure(figsize=(6, 4))  
plt.plot(x, linear(x), label="Linear", color="red")  
plt.title("Linear Activation Function")  
plt.xlabel("Input")  
plt.ylabel("Output")  
plt.legend()  
plt.grid()  
plt.show()
```

1. Sigmoid Function

Source code and Output:-

```
import numpy as np  
import matplotlib.pyplot as plt  
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))  
x = np.linspace(-10, 10, 100)  
plt.figure(figsize=(6, 4))  
plt.plot(x, sigmoid(x), label="Sigmoid", color="green")
```

```
plt.title("Sigmoid Activation Function")
plt.xlabel("Input")
plt.ylabel("Output")
plt.legend()
plt.grid()
plt.show()
```

2. Tanh Activation Function

Source code and Output:-

```
def tanh(x):
    return np.tanh(x)
x = np.linspace(-10, 10, 100)
plt.plot(x, tanh(x), label="tanh", color="orange")
plt.title("Activation Function: tanh")
plt.xlabel("Input (x)")
plt.ylabel("Output (tanh(x))")
plt.legend()
plt.grid()
plt.show()
```

3. ReLU (Rectified Linear Unit) Function

Source code and Output:-

```
import numpy as np
import matplotlib.pyplot as plt
def RELU(x):
    x1 = []
    for i in x:
        if i < 0:
            x1.append(0)
        else:
            x1.append(i)
    return np.array(x1)
```

```
x = np.linspace(-10, 10, 100)
plt.plot(x, RELU(x), label="RELU", color = "pink")
plt.title("Activation Function: ReLU")
plt.xlabel("Input (x)")
plt.ylabel("Output (ReLU(x))")
plt.legend()
plt.grid()
plt.show()
```

1. Softmax Function

Source code and Output:-

```
def softmax(x):
    exp_x = np.exp(x)
    return exp_x / np.sum(exp_x)

x = np.linspace(-10, 10, 100)
y = softmax(x)

plt.plot(x, y, label="Softmax", color="blue")
plt.title("Activation Function: Softmax")
plt.xlabel("Input (x)")
plt.ylabel("Output (Softmax(x))")
plt.legend()
plt.grid()
plt.show()
```

Binary Activation Function

Source code and Output:-

```

import numpy as np

import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)

def binaryStep(x):

    return np.heaviside(x, 1)

plt.figure(figsize=(6, 4))

plt.plot(x, binaryStep(x), label="Binary Step")

plt.title("Binary Step Activation Function")

plt.xlabel("Input")

plt.ylabel("Output")

plt.legend()

plt.grid()

plt.show()

```

Aim : Implementation of Hebbian Learning Rule

Source code and Output:-

```

import numpy as np

w = np.array([1, -1, 0, 0.5]).transpose()

xi = [
    np.array([1, -2, 1.5, 0]).transpose(),
    np.array([1, -0.5, -2, -1.5]).transpose(),
    np.array([0, 1, -1, 1.5]).transpose()
]

c = 1

iterations = 3

for iteration in range(iterations):

```

```

for i in range(len(xi)):
    net = np.dot(w.transpose(), xi[i])
    Fnet = np.sign(net)
    dw = c * Fnet * xi[i]
    w = w + dw
print("Final weight matrix:", w)
print("Total iterations:", iterations)

```

Final weight matrix: [1. -8.5 13.5 0.5]
 Total iterations: 3

Aim :- Implementation of Perceptron Learning Rule

Source code and Output:-

```

W = np.array([1, -1, 0, 0.5]) Xi = [
    np.array([1, -2, 0, -1]),
    np.array([0, 1.5, -0.5, -1]),
    np.array([-1, 1, 0.5, -1])
]
d = np.array([-1, -1, 1])
c = 0.1
iterations = 10
for _ in range(iterations):
    for i in range(len(Xi)):
        net = np.dot(W, Xi[i])
print(f"Net: {net}")

```

```

Fnet = 1 if net >= 0 else -1

error = d[i] - Fnet

print(f"Error: {error}")

if error != 0:

    W += c * error * Xi[i]

print(f"Updated Weights: {W}\n")

```

Output:-

Net: 2.5

Error: -2

Updated Weights: [0.8 -0.6 0. 0.7]

Net: -1.5999999999999999

Error: 0

Updated Weights: [0.8 -0.6 0. 0.7]

Net: -2.0999999999999996

Error: 2

Updated Weights: [0.6 -0.4 0.1 0.5]

Net: 0.8999999999999999

Error: -2

Updated Weights: [4.00000000e-01 5.55111512e-17 1.00000000e-01
7.00000000e-01]

Aim: Implementation of Delta Learning Rule

Source code and Output:-

```
import pandas as pd
import numpy as np

w = np.array([1,-1,0,0.5]).transpose()

Xi =
[np.array([1,-2,0,-1]).transpose(),np.array([0,1.5,0.5,-1]).transpose(),np.array
([-1,1,0.5,-1]).transpose()]

d =[-1,-1,1]

c=0.1

Error=1

iteration=0

i=0

j=0

for i in range(len(Xi)):

    net = sum(w.transpose()*Xi[i])

    print("Net : {}".format(net))

    o = (2/(1+np.exp(-1*net)))-1

    o_ = (0.5)*(1-(o**2))

    error = d[i]-o

    print("Error")

    print(round(error,1))

    Error = (round(error,1))

    dw = c*error*o_*Xi[i]
```

```
w=w+dw  
print("Weight Matrix: {}".format(w))
```

Output :-

Net : 2.5

Error

-1.8

Weight Matrix: [0.97408569 -0.94817138 0. 0.52591431]

Net : -1.948171379043783

Error

-0.2

Weight Matrix: [0.97408569 -0.95634441 -0.00272434 0.531363]

Net : -2.463155269369176

Error

1.8

Weight Matrix: [0.94742711 -0.92968583 0.01060494 0.50470442]

Aim: Implementation of Single Discrete Perceptron Training Algorithm

Source code and Output:-

```
import pandas as pd  
import numpy as np  
  
w= np.array([1,-1,0,0.5]).transpose()  
  
Xi=[np.array([1,-2,0,-1]).transpose(),np.array([0,1.5,0.5,-1]).transpose(),n  
p.array([-1,1,0.5,-1]).transpose()]  
  
d=[-1,-1,1]
```

```
c=1
Error=1
iteration=0
i=0
j=0
error=0
while(Error !=0.0):
    net=sum(w.transpose()*Xi[i])
    o=1
    if net<0:
        o=-1
    err = d[i] - o
    error += 0.5 * (err ** 2)
    print(round(error,1))
    Error=round(error,1)
    dw=c*error*Xi[i]
    w=w+dw
    iteration+=1
    i+=1
    if i>2:
        i=0
        j+=1
        error=0
```

```

if Error==0.0:

    break

print("Final Weight Matrix : {}".format(w))

print("Cycle epoch counter: {}".format(j))

print("Iteration : {}".format(iteration))

```

2.0

2.0

4.0

2.0

4.0

4.0

0.0

Final Weight Matrix : [-3. 8. 7. -17.5]

Cycle epoch counter: 2

Iteration : 7

Aim : Implementation of Single Continuous Perceptron Training Algorithm

Source code and Output:-

```

import numpy as np

import pandas as pd

w=np.array([1,-1,0,0.5])

xi=[np.array([1,-2,0,-1]).transpose(),np.array([0,1.5,0.5,-1]).transpose(),np.array([1,0.5,1.1,0])]

d = [-1,-1,1]

c = 1

```

```

Error = 1

iteration = 0

i = 0

j = 0

error = 0

while(Error != -0.0):

    net = sum(w.transpose()*xi[i])

    o=(2/(1+np.exp(-1*net)))-1

    o_ = (0.5)*(1-(o**2))

    error = error+(0.5)*((d[i]-o)**2)

    print(round(error,1))

    Error = round(error,1)

    dw = c*(d[i]-o)*o_*xi[i]

    w=w+dw

    iteration =1

    i+=1

    if i> 2:

        i=0

        j+=1

        error=0

        if Error ==-0.0:

            break

print("Final Weight Matrix: {}".format(w))

print("Cycle epoch counter: {}".format(j))

print("Iteration: {}".format(iteration))

```

Output:-

1.7

1.8

2.1

1.2

1.5

1.7

0.1

0.2

0.3

0.0

Final Weight Matrix:[0.8066503 0.38217628 0.55448015 2.25624372]

Cycle epoch counter:3

Aim : Implementation of Perceptron as Classifier as Iris Data

Source code and Output:-

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, classification_report

import random

from sklearn.linear_model import Perceptron

iris = load_iris()

iris.target_names

target = (iris.target==0).astype(np.int8)
```

```
print(target)

datasets = train_test_split(iris.data, target, test_size=0.2)

X_train, X_test, y_train, y_test = datasets

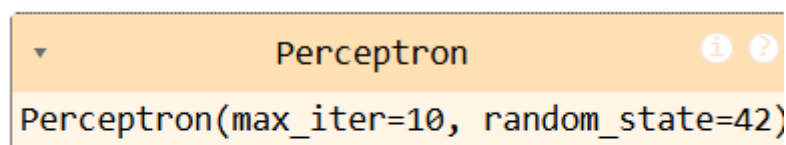
[[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
  1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0]
```

```
from sklearn.linear_model import Perceptron

p = Perceptron(random_state=42, max_iter=10, tol=0.001)

p.fit(X_train, y_train)

Perceptron(max_iter=10, random_state=42)
```



```
Perceptron(max_iter=10, random_state=42)
```

```
import random

sample = random.sample(range(len(X_train)), 10)

for i in sample:
    print(i, p.predict([X_train[i]]))

58 [0]
108 [0]
40 [1]
55 [1]
```

115 [1]

89 [0]

51 [0]

0 [0]

86 [0]

102 [1]

```
from sklearn.metrics import classification_report
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	24
1	1.00	1.00	1.00	6
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Aim: Implementation of Multilayer Perceptron as classifier of Breast Cancer Data

Source code and Output:-

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

cancer_data = load_breast_cancer()

X,y = cancer_data.data,cancer_data.target
```



```

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test= scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(64,32),max_iter=1000,random_state
=42)

mlp.fit(X_train,y_train)

y_pred = mlp.predict(X_test)

accuracy = accuracy_score(y_test,y_pred)

print(f"Accuracy: {accuracy}")

print("Classification Report:")

print(classification_report(y_test,y_pred))

```

Accuracy: 0.9736842105263158

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.96	43
1	0.97	0.99	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Aim: Implementation of Multilayer Perceptron Model using TensorFlow and Keras.

Source code and Output:-

```
import tensorflow as tf

import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Flatten, Dense, Activation

import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = X_train.astype('float32') / 255.0

x_test = X_test.astype('float32') / 255.0


print("Feature matrix:", x_train.shape)

print("Feature matrix:", x_test.shape)

print("Target matrix:", y_train.shape)

print("Target matrix:", y_test.shape)


# Display some images

fig, ax = plt.subplots(10, 10, figsize=(10, 10))

k = 0

for i in range(10):

    for j in range(10):
```

```
ax[i, j].imshow(x_train[k].reshape(28, 28), cmap='gray')

ax[i, j].axis('off')

k += 1

plt.show()

model = Sequential([

    Flatten(input_shape=(28, 28)),

    Dense(256, activation='sigmoid'),

    Dense(128, activation='sigmoid'),

    Dense(10, activation='sigmoid')

])

model.compile(

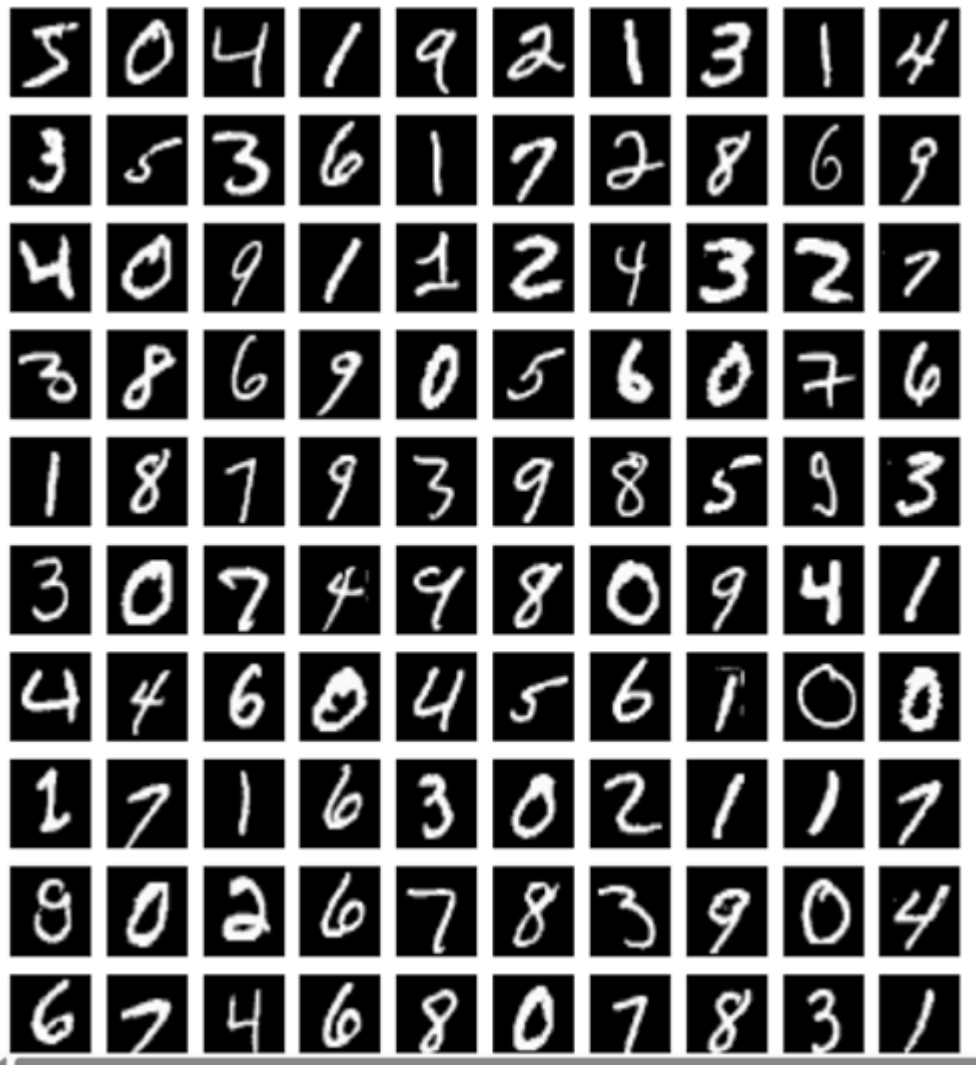
    optimizer='adam',

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)

model.summary()
```



Feature matrix: (60000, 28, 28)

Feature matrix: (10000, 28, 28)

Target matrix: (60000,)

Target matrix: (10000,)

Total params: 235,146 (918.54 KB)

Trainable params: 235,146 (918.54 KB)

Non-trainable params: 0(0.00 B)

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None , 784)	0
dense (Dense)	(None , 256)	200,960
dense_1 (Dense)	(None , 128)	32,896
dense_2 (Dense)	(None , 10)	1,290

Aim: Implementation of Convolutional Neural Network(CNN)

Source code and Output:-

```
import tensorflow as tf

from tensorflow.keras import layers, models, datasets

import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) =
datasets.cifar10.load_data()

train_images, test_images = train_images/255.0, test_images/255.0

class_names =
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))

for i in range(25):

    plt.subplot(5,5,i+1)

    plt.xticks([])

    plt.yticks([])

    plt.grid(False)

    plt.imshow(train_images[i])
```

```

plt.xlabel(class_names[train_labels[i][0]])
plt.show()

model = models.Sequential()

model.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)))

model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64,(3,3),activation='relu'))

model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64,(3,3),activation='relu'))

model.summary()

model.add(layers.Flatten())

model.add(layers.Dense(64,activation='relu'))

model.add(layers.Dense(10))

model.summary()

model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategorical
Crossentropy(from_logits=True),metrics=['accuracy'])

history =
model.fit(train_images,train_labels,epochs=10,validation_data=(test_im
ages,test_labels))

```

```

plt.plot(history.history['accuracy'],label='accuracy')
plt.plot(history.history['val_accuracy'],label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5,1])
plt.legend(loc='lower right')

test_loss,test_acc = model.evaluate(test_images,test_labels,verbose=2)

print(test_acc)

```

Aim: Implementation of Recurrent Neural Network(RNN)

Source code and Output:-

```

!pip install tensorflow==2.2
!pip install keras

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, LSTM, Dropout

mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train/255.0
x_test = x_test/255.0

print(x_train.shape)

```

```
print(x_train[0].shape)

model = Sequential()

model.add(LSTM(128, input_shape=(x_train.shape[1:]), activation='relu',
return_sequences=True))

model.add(Dropout(0.2))

model.add(LSTM(128, activation='relu'))

model.add(Dropout(0.1))

model.add(Dense(32, activation='relu'))

model.add(Dropout(0.2))

model.add(Dense(10, activation='softmax'))

opt = tf.keras.optimizers.Adam(learning_rate=0.651, decay=1e-6)

model.compile(loss='sparse_categorical_crossentropy', optimizer=opt,
metrics=['accuracy'])

model.fit(x_train,y_train,epochs=3,validation_data=(x_test, y_test))

score = model.evaluate(x_test, y_test, verbose=0)

print('Test loss:', score[0])

print('Test accuracy:', score[1])

model.fit(x_train, y_train, epochs=3, validation_data=(x_test, y_test))
```